

Ablaufplan. KI-gestützte Quanten-Transpilation

≈ 40 Min · Folien Englisch, Vortrag Deutsch · 2 Demos in VS Code (Kernel „cwq“, offline, kein Token)

Zeit	Folien	Abschnitt	Kernsatz / Hinweis
0-4	2-3	Was ist Transpilation?	„Transpilation ändert nicht, WAS gerechnet wird, sondern WIE es auf dieser Maschine läuft.“ → Animation auf Folie 3 laufen lassen
4-6	4	Die 6 Stufen (Überblick)	„Sechs Stufen, ein Zwei-Qubit-Schaltkreis. Gleich live im Notebook.“
6-16	5 ▶	DEMO 1: Bell durch 6 Stufen (VS Code · Notebook 1)	init→layout→routing→translation→optimization→scheduling. Routing-Spiel: 1 SWAP = 3 CNOTs. Aus 1 Zwei-Qubit-Gatter werden 4.
16-19	6-7	KI in der Transpilation: das Spiel + RL	„Statt fester Regeln lernt ein Modell, am günstigsten zu verdrahten.“ 4 CNOTs (96 Pkt.) schlägt 2 SWAPs + 1 CNOT (93 Pkt.).
19-26	8 ▶	DEMO 2: KI-Transpiler (VS Code · Notebook 2)	QPM vs QTS vs SQR, vorgerechnet. Reihenfolge: Spiel → Fig 16 → Fig 18 (die drei Gruppen).
26-40	9-16	Meine Thesis + Abschluss	Chip-Wechsel HH→Quadratgitter · Forschungsfrage · 69 Modelle (Subgraph-Extraktion) · bis 46 % · 3 Regime · Grenze · squarebench. Danke + Fragen.

Schlüsselzahlen (auswendig)

- bis 46 % weniger Zwei-Qubit-Tiefe auf strukturierten Schaltungen
- BV: 429 → 234 CZ (−45,5 %) · SU2-89: Tiefe −45,9 %
- Quadratgitter allein: 30-45 % flacher als Heavy-Hex (Ausnahme SU2-89: +100 %, ungerader Ring)
- SQR schlägt QTS: −13...24 % Tiefe / −13...28 % Gatter (Spitze bei 6 Qubits)
- 69 nachtrainierte Modelle · 3 Regime: Besser / Gleich / Schlechter

Demo-Hinweise

- Notebooks sind vorausgeführt. Nur scrollen & erzählen (Kernel „cwq“, offline, kein Token nötig).
- GIF (Folie 3) animiert nur im Präsentationsmodus. Volle Detailversion: slides/Transpilation_full.pptx.
- Sprechernotizen (Deutsch, mit fettem Kernsatz) auf jeder Folie: Ansicht ▶ Notizen.